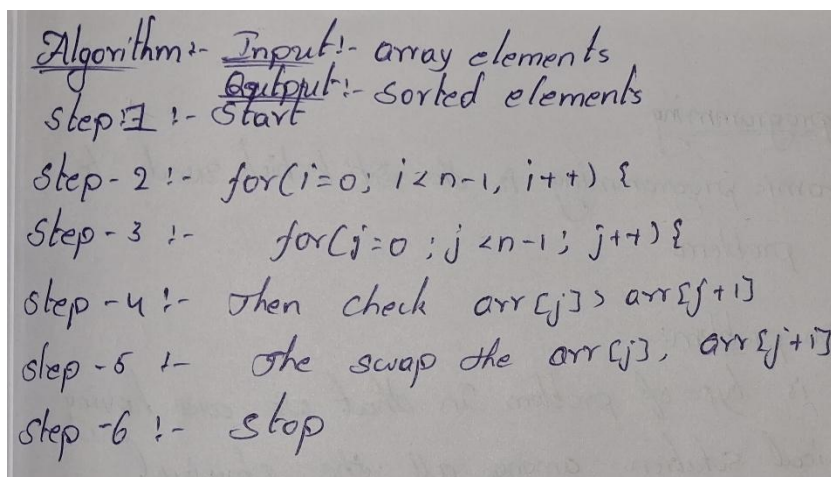


Practical 1

Aim: To Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort

Bubble sort

Algorithm:



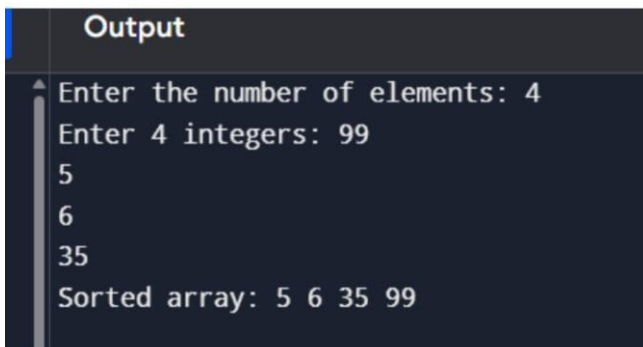
Algorithm:-
Input:- array elements
Output:- sorted elements
Step-1 :- Start
Step-2 :- for($i=0$; $i < n-1$; $i++$) {
Step-3 :- for($j=0$; $j < n-1$; $j++$) {
Step-4 :- Then check $arr[j] > arr[j+1]$
Step-5 :- the swap the $arr[j]$, $arr[j+1]$
Step-6 :- stop

Code:

```
#include <iostream>
#include <vector>
using namespace std;
void bubbleSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; ++i) {
        for (int j = 0; j < n - i - 1; ++j) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```

```
    }  
    }  
}  
  
int main() {  
    int size;  
    cout << "Enter the number of elements: ";  
    cin >> size;  
    vector<int> arr(size);  
    cout << "Enter " << size << " integers: ";  
    for (int i = 0; i < size; ++i) {  
        cin >> arr[i];  
    }  
    bubbleSort(arr);  
    cout << "Sorted array: ";  
    for (int i = 0; i < size; ++i) {  
        cout << arr[i] << " ";  
    }  
  
    cout << endl;  
    return 0;  
}
```

Output:



```
Output  
Enter the number of elements: 4  
Enter 4 integers: 99  
5  
6  
35  
Sorted array: 5 6 35 99
```

Time complexity:

Time Complexity of Bubble Sort

```

for(i=0; i < n-1; i++) {
    for(j=0; j < n-i-1; j++) {
        if(arr[j] > arr[j+1]) {
            temp = arr[j]
            arr[j] = arr[j+1]
            arr[j+1] = temp
        }
    }
}

```

$T(n) = (n-1)(n-1) + 4$
 $= n^2 - 2n + 4$
 $T(n) = O(n^2)$

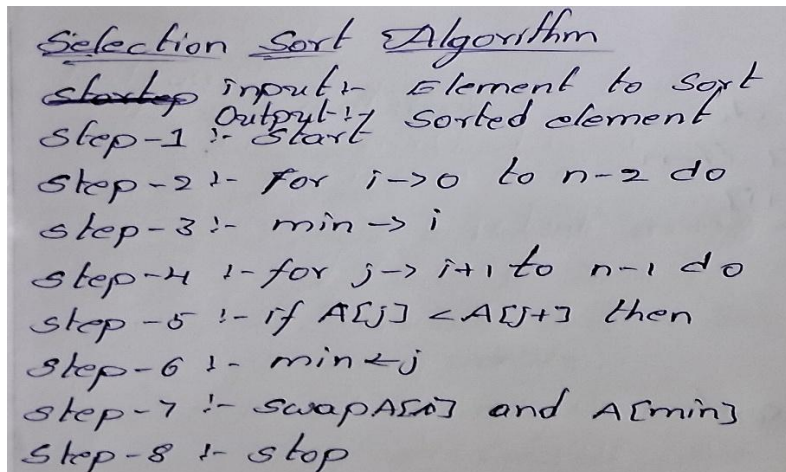
i) Best case: (The array is sorted)
no. of comparisons $n-1$
 $T(n) = n-1$
 $T(n) = O(n)$

ii) Average case
 $(n-1) + (n-2) + \dots + 1$
 $1 + 2 + 3 + \dots + (n-1)$
 $n(n-1)/2$
 $T(n) = n(n-1)/2$
 $T(n) = O(n^2)$

iii) Worst case:
 $(n-1) + (n-2) + (n-3) + \dots + 1$
 $= n(n-1)/2$
 $= (n^2 - n)/2$
 $T(n) = O(n^2)$

Selection sort

Algorithm:



Selection Sort Algorithm
Start Input:- Element to sort
Output:- Sorted element
Step-1 :- Start
Step-2 :- For $i \rightarrow 0$ to $n-2$ do
Step-3 :- $\text{min} \rightarrow i$
Step-4 :- For $j \rightarrow i+1$ to $n-1$ do
Step-5 :- If $A[j] < A[\text{min}]$ then
Step-6 :- $\text{min} \leftarrow j$
Step-7 :- Swap $A[i]$ and $A[\text{min}]$
Step-8 :- Stop

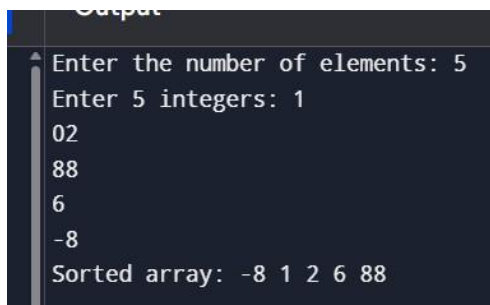
Code :

```
#include <iostream>
#include <vector>
void selectionSort(std::vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; ++i) {
        int minIndex = i;
        for (int j = i + 1; j < n; ++j) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        if (minIndex != i) {
            std::swap(arr[i], arr[minIndex]);
        }
    }
}

int main() {
    int n;
    std::cout << "Enter the number of elements: ";
    std::cin >> n;
    if (n <= 0) {
        return 0;
    }
}
```

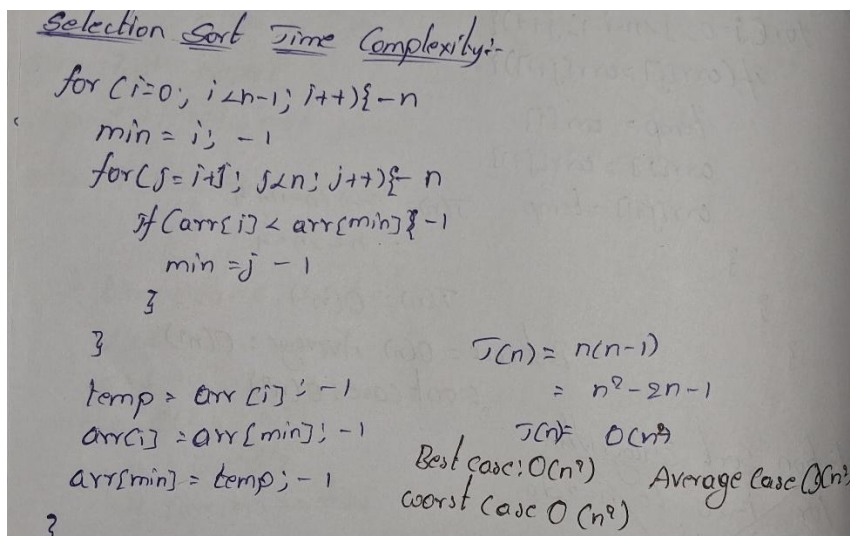
```
std::vector<int> arr(n);
std::cout << "Enter " << n << " integers: ";
for (int i = 0; i < n; ++i) {
    std::cin >> arr[i];
}
selectionSort(arr);
std::cout << "Sorted array: ";
for (int i = 0; i < n; ++i) {
    std::cout << arr[i] << " ";
}
return 0;
}
```

Output:



```
Enter the number of elements: 5
Enter 5 integers: 1
02
88
6
-8
Sorted array: -8 1 2 6 88
```

Time complexity:



Selection Sort Time Complexity:-

```
for (i=0; i < n-1; i++) { - n
    min = i; - 1
    for (j=i+1; j < n; j++) { - n
        if (arr[i] < arr[min]) { - 1
            min = j; - 1
        }
    }
}

temp = arr[i]; - 1
arr[i] = arr[min]; - 1
arr[min] = temp; - 1
}
```

$T(n) = \frac{n(n-1)}{2}$
 $= \frac{n^2 - 2n - 1}{2}$
 $T(n) = O(n^2)$
 Best case: $O(n^2)$ Average case: $O(n^2)$
 Worst case: $O(n^2)$

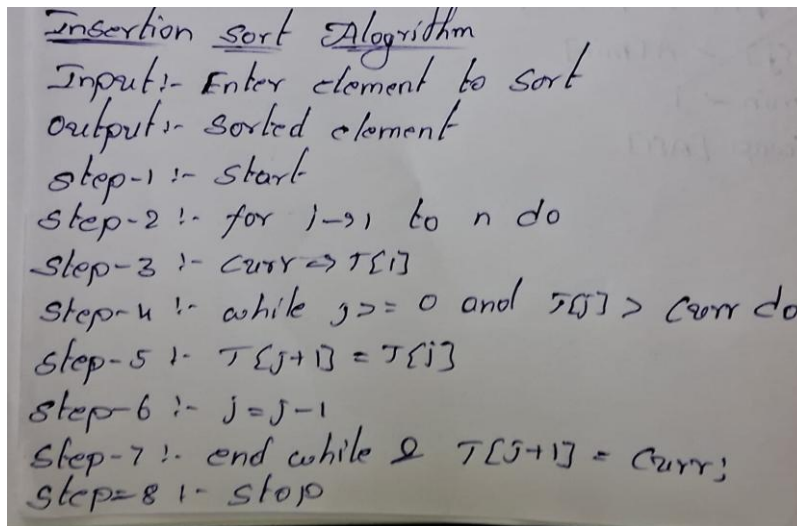
i) Best Case
Perform all comparison
 $(n-1) + (n-2) + \dots + 2 + 1$
 $T(n) = n(n-1)/2$
 $= O(n^2)$

ii) Average Case
check every pass
 $T(n) = (n-1) + (n-2) + \dots + 1$
 $= n(n-1)/2$
 $T(n) = O(n^2)$

iii) Worst Case
all comparison
 $T(n) = O(n^2)$

Insertion sort

Algorithm:



Insertion sort Algorithm
Input:- Enter element to sort
Output:- Sorted element
Step-1 :- Start
Step-2 :- for $j=1$ to n do
Step-3 :- $curr \rightarrow T[j]$
Step-4 :- while $j \geq 0$ and $T[j] > curr$ do
Step-5 :- $T[j+1] = T[j]$
Step-6 :- $j = j - 1$
Step-7 :- end while & $T[j+1] = curr$;
Step-8 :- stop

Code:

```
#include <iostream>
#include <vector>
using namespace std;
void insertionSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 1; i < n; ++i) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}
int main() {
    int n;
    cout << "Enter the number elements:";
    cin >> n;
    if (n <= 0) {
        return 0;
    }
    vector<int> arr(n);
    cout << "Enter " << n << " integers: ";
    for (int i = 0; i < n; ++i) {
```



```

        cin >> arr[i];
    }
    insertionSort(arr);
    cout << "Sorted array: ";
    for (int i = 0; i < n; ++i) {
        cout << arr[i] << " ";
    }
    cout << endl;
    return 0;
}

```

Output:

```

Enter the number of elements: 5
Enter 5 integers: 3
55
89
20
6
Sorted array: 3 6 20 55 89

```

Time complexity:

Insertion Sort Time Complexity :-

Best Case
 $\text{for } i=1; i \leq n; i++ \{$
 $\text{key} = \text{arr}[i];$
 $j = i-1;$
 $\text{while } (j > 0 \text{ \& \& } \text{arr}[j] > \text{key}) \{$
 $\text{arr}[j+1] = \text{arr}[j];$
 $j = j-1;$
 $\}$
 $\text{arr}[j+1] = \text{key};$
 $\}$

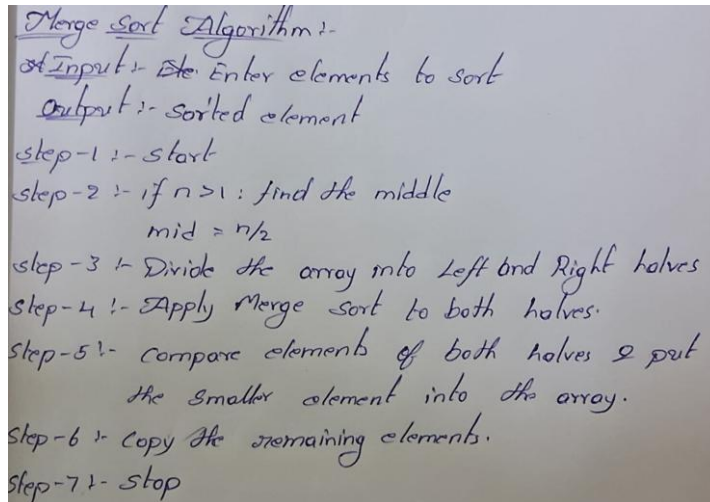
Average Case :- $1+2+3+\dots+(n-1)$
 $T(n) = n(n-1)/2$
 $T(n) = O(n^2)$

Worst Case :- $(n-1) + (n-2) + \dots + 1$
 $T(n) = n(n-1)/2$
 $= O(n^2)$

Best case: $T(n) = O(n)$
 Average case: $O(n^2)$
 Worst case: $O(n^2)$

Merge Sort

Algorithm:



Merge Sort Algorithm:-
Input:- Enter elements to sort
Output:- Sorted element
Step-1:- Start
Step-2:- if $n > 1$: find the middle
 $mid = n/2$
Step-3:- Divide the array into Left and Right halves
Step-4:- Apply Merge sort to both halves.
Step-5:- Compare elements of both halves & put the smaller element into the array.
Step-6:- Copy the remaining elements.
Step-7:- Stop

Code:

```
#include <iostream>
#include <vector>
using namespace std;
void merge(vector<int>& arr, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;
    vector<int> L(n1);
    vector<int> R(n2);
    for (int i = 0; i < n1; ++i) {
        L[i] = arr[left + i];
    }
    for (int j = 0; j < n2; ++j) {
        R[j] = arr[mid + 1 + j];
    }
    int i = 0;
    int j = 0;
    int k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            ++i;
        }
    }
```

```
        else {
            arr[k] = R[j];
            ++j;
        }
        ++k;
    }
    while (i < n1) {
        arr[k] = L[i];
        ++i;
        ++k;
    }
    while (j < n2) {
        arr[k] = R[j];
        ++j;
        ++k;
    }
}

void mergeSort(vector<int>& arr, int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

int main() {
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;
    if (n <= 0) {
        return 0;
    }
    vector<int> arr(n);
    cout << "Enter " << n << " integers: ";
    for (int i = 0; i < n; ++i) {
        cin >> arr[i];
```

```

}
mergeSort(arr, 0, n - 1);
cout << "Sorted array: ";
for (int i = 0; i < n; ++i) {
    cout << arr[i] << " ";
}
cout << endl;
return 0;
}

```

Output:

```

Output
Enter the number of elements: 5
Enter 5 integers: 66
4
7
45
5
Sorted array: 4 5 7 45 66

```

Time Complexity

Merge Sort Time Complexity:-

```

mergeSort(int arr[], left, right){
    if(left > right){
        return;
    }
    int m = (left + right) / 2;
    mergeSort(arr, left, m);
    mergeSort(arr, m + 1, right);
    merge(arr, left, m, right); // O(n)
}

```

Best Case:-

$$T(n) = O(n) \times O(\log_2 n)$$

$$= O(n \log n)$$

Average Case:-

$$T(n) = n \times \log_2 n$$

$$T(n) = O(n \log n)$$

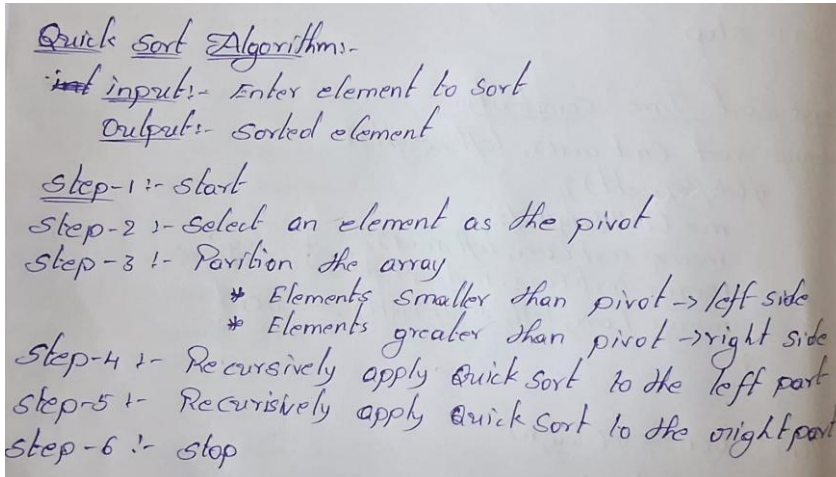
Worst Case:-

$$T(n) = n \times \log_2 n$$

$$= O(n \log n)$$

Quicksort:

Algorithm:



Quick Sort Algorithms:-
input:- Enter element to sort
Output:- Sorted element

Step-1:- start
Step-2 :- select an element as the pivot
Step-3 :- Partition the array
 * Elements smaller than pivot $\rightarrow$ left side
 * Elements greater than pivot $\rightarrow$ right side
Step-4 :- Recursively apply Quick Sort to the left part
Step-5 :- Recursively apply Quick Sort to the right part
Step-6 :- stop

Code:

```
#include <iostream>

#include <vector>

using namespace std;

void swap(int& a, int& b) {
    int temp = a;
    a = b;
    b = temp;
}

int partition(vector<int>& arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
```

```
}  
swap(arr[i + 1], arr[high]);  
return i + 1;  
}  
void quickSort(vector<int>& arr, int low, int high) {  
    if (low < high) {  
        int pi = partition(arr, low, high);  
        quickSort(arr, low, pi - 1);  
        quickSort(arr, pi + 1, high);  
    }  
}  
int main() {  
    int n;  
    cout << "Enter number of elements: ";  
    cin >> n;  
    vector<int> arr(n);  
    cout << "Enter " << n << " elements: ";  
    for (int i = 0; i < n; i++) {  
        cin >> arr[i];  
    }  
    quickSort(arr, 0, n - 1);  
    cout << "Sorted array: ";  
    for (int i = 0; i < n; i++) {  
        cout << arr[i] << " ";  
    }  
    cout << endl;  
    return 0;  
}
```

Output:

```

Output
Enter number of elements: 5
Enter 5 elements: 99

8
88
52
56
Sorted array: 8 52 56 88 99

```

Time complexity

Quick Sort

Best case

```

if (low < high) { - 1
    Pivot = arr[high] - 1
    i = low - 1
    for (j = low; j < high; j++) { - n
        if (arr[j] < pivot) { - 1
            {
                j++;
                temp = arr[i] - 1
                arr[i] = arr[j] - 1
                arr[j] = temp - 1
            }
        }
    }
    temp = arr[i+1] - 1
    arr[i+1] = arr[high] - 1
    arr[high] = temp - 1
    P = i+1
    QuickSort(arr, low, P-1) - 1
    QuickSort(arr, P+1, high) - 1
}

```

Average Case: divide into balance parts

$$n + n/2 + n/2 + n/4 + n/4 + \dots$$

$$T(n) = n \times \log_2 n$$

$$T(n) = O(n \log n)$$

Worst case

$$n + (n-1) + (n-2) + \dots + 1$$

$$T(n) = n(n-1)/2$$

$$= (n^2 - n)/2$$

$$T(n) = O(n^2)$$

~~$T(n) = O(n)$~~ Best case = $O(n \log n)$ Average case = $O(n \log n)$

Conclusion:

From this practical, various sorting algorithms such as Selection Sort, Bubble Sort, Insertion Sort, Merge Sort, and Quick Sort were studied and implemented successfully. Selection, Bubble, and Insertion Sort are simple and easy to understand, while Merge Sort and Quick Sort provide better performance for large datasets. The time complexity of these algorithms varies from $O(n)$ to $O(n^2)$, with Merge Sort having $O(n \log n)$ time complexity in all cases and Quick Sort having an average time complexity of $O(n \log n)$. Thus, the practical helped in understanding different sorting techniques, their working principles, and their time and space complexities.